Module: 1
Practical 1

Design a simple rational agent by defining PEAS for a Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (Fully / Partially Observable, Deterministic / Stochastic, Episodic / Sequential).Implement a simple table-driven agent in Python.

Code:

```
rooms = {}

rooms['A'] = input("Enter status of Room A(Clean/Dirty):");
rooms['B'] = input("Enter status of Room B(Clean/Dirty):");

if rooms['A'] == "Clean" and rooms['B'] == "Clean":
   print("\nAll Rooms are clean. Task Completed")
else:
   if rooms['A'] == "Dirty":
      print("\nRoom A is Dirty")
      print("Action:Suck")
      rooms['A'] == "Clean"
      print("Room A is Cleaned")
   else:
      print("\nRoom A is Already clean")

   if rooms['B'] == "Dirty":
      print("\nRoom B is Dirty")
      print("Action:Suck")
      rooms['B'] == "Clean"
      print("Room B is Cleaned")
   else:
      print("Room B is Already Clean")

   print("\nAll Rooms are clean. Task Completed")
```

File Edit Format Run Options Window Help

```
rooms={}
rooms['A']=input("Enter status of Room A(Clean/Dirty):");
rooms['B']=input("Enter status of Room B(Clean/Dirty):");

if rooms['A']=="Clean" and rooms['B']=="Clean":
    print("\nAll Rooms are clean. Task Completed")
else:
    if rooms['A']=="Dirty":
        print("\nRoom A is Dirty")
        print("Action:Suck")
        rooms['A']=="Clean"
        print("Room A is Cleaned")
    else:
        print("\nRoom A is Already clean")
    if rooms['B']=="Dirty":
        print("\nRoom B is Dirty")
        print("Action:Suck")
        rooms['B']=="Clean"
        print("Room B is Cleaned")
    else:
        print("Room B is Already Clean")
    print("\nAll Rooms are clean. Task Completed")

```

Output:

```
>
========== RESTART: C:/Users/Admin/Documents/Rooms.py
Enter status of Room A(Clean/Dirty):Dirty
Enter status of Room B(Clean/Dirty):Dirty

Room A is Dirty
Action: Suck
Room A is Cleaned

Room B is Dirty
Action: Suck
Room B is Cleaned

All rooms are clean. Task completed.
>
```

Practical 2

Given an initial configuration of the 8-Puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

Code:

```
from collections import deque

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()


def get_neighbors(state):
    neighbors = []
    blank_index = state.index(0)
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, position_change in moves.items():
        new_index = blank_index + position_change

        if move == "Up" and blank_index < 3:
            continue

        if move == "Down" and blank_index > 5:
            continue

        if move == "Left" and blank_index % 3 == 0:
            continue

        if move == "Right" and blank_index % 3 == 2:
            continue

        new_state = list(state)
        new_state[blank_index], new_state[new_index] = new_state[new_index],
new_state[blank_index]
        neighbors.append((tuple(new_state), move))
```

```
    return neighbors


def bfs(initial_state, goal_state):
    queue = deque()
    queue.append((initial_state, []))
    visited = set()
    visited.add(initial_state)

    while queue:
        current_state, path = queue.popleft()

        if current_state == goal_state:
            return path

        for neighbor, move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [(move, neighbor)]))

    return None


initial_state = (1, 2, 3,
                 4, 0, 6,
                 7, 5, 8)

goal_state = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

print("Initial State:")
print_state(initial_state)

print("Goal State:")
print_state(goal_state)

solution = bfs(initial_state, goal_state)

if solution:
    print("Shortest sequence of moves:")
    current_step = 1
```

```
    for move, state in solution:
        print("Step", current_step, "- Move:", move)
        print_state(state)
        current_step += 1

    print("Goal reached successfully.")
    print("Total number of moves:", len(solution))

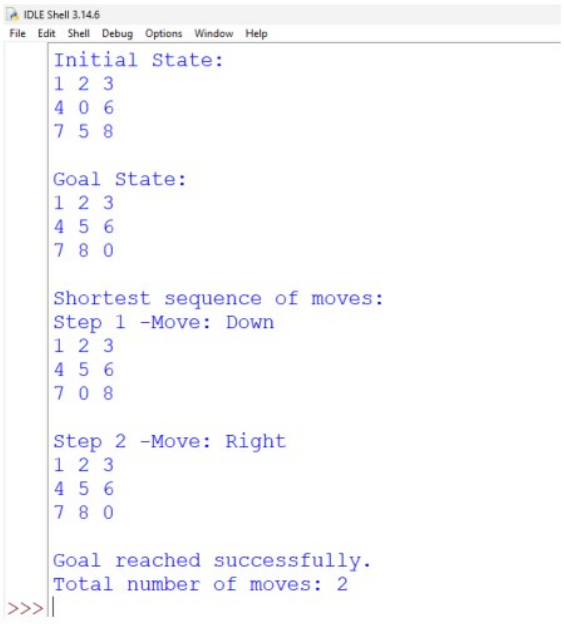
else:
    print("No solution found.")
```

```
File  Edit  Format  Run  Options  Window  Help
from collections import deque

def print_state(state):
    for i in range (0, 9, 3):
        print(state[i], state[i+1], state[i+ 2])
    print()

def get_neighbors(state):
    neighbors=[]
    blank_index= state.index(0)
    moves = {
        "Up":-3,
        "Down":3,
        "Left":-1,
        "Right":1
        }
    for move, position_change in moves.items():
        new_index = blank_index + position_change
        if move=="Up" and blank_index<3:
            continue
        if move == "Down" and blank_index > 5:
            continue
        if move == "Left" and blank_index % 3== 0:
            continue
        if move == "Right" and blank_index % 3 == 2:
            continue
        new_state = list(state)
        new_state[blank_index], new_state[new_index]=new_state[new_index], new_state[blank_index]
        neighbors.append((tuple(new_state), move))
    return neighbors
def bfs(initial_state,goal_state):
    queue = deque()
    queue.append((initial_state, []))
    visited = set()
    visited.add(initial_state)
    while queue:
        current_state, path = queue.popleft()
        if current_state==goal_state:
            return path
        for neighbor,move in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [(move, neighbor)]))
    return None
initial_state=(1, 2, 3,
                  4, 0, 6,
                  7, 5, 8)
goal_state=(1, 2, 3,
               4, 5, 6,
               7, 8, 0)
print("Initial State:")
print_state(initial_state)
print ("Goal State:")
print_state(goal_state)
solution = bfs(initial_state,goal_state)
if solution:
    print ("Shortest sequence of moves:")
    current_step = 1
    for move,state in solution:
        print("Step",current_step, "- Move:", move)
        print_state(state)
        current_step+= 1
    print("Goal reached successfully.")
    print("Total number of moves:",len(solution))
else:
    print("No solution found.")
```

Output:


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Initial State:
1 2 3
4 0 6
7 5 8
Goal State:
1 2 3
4 5 6
7 8 0
Shortest sequence of moves:
Step 1 -Move: Down
1 2 3
4 5 6
7 0 8
Step 2 -Move: Right
1 2 3
4 5 6
7 8 0
Goal reached successfully.
Total number of moves: 2
>>>

Practical 3

Given two water jugs of 4 litres and 3 litres (capacity), write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

Code:

```
j1 = int(input("Enter Capacity of jug 1:"))
j2 = int(input("Enter Capacity of jug 2:"))
goal = int(input("Enter Target Amount:"))

visited = set()

def dfs(x, y, path):
    if x == goal or y == goal:
        path.append((x, y))
        print("\nGoal achieved Successfully!")
        print("Solution Path:")
        for state in path:
            print(state)
        return True

    if (x, y) in visited:
        return False

    visited.add((x, y))
    path.append((x, y))

    next_state = [
        (j1, y),
        (x, j2),
        (0, y),
        (x, 0),

        (x - min(x, j2 - y),
         y + min(x, j2 - y)),

        (x + min(y, j1 - x),
         y - min(y, j1 - x))
    ]

    for state in next_state:
        if dfs(state[0], state[1], path.copy()):
            return True
```

```
    return False


if not dfs(0, 0, []):
    print("No solution found")
```

File Edit Format Run Options Window Help

```
j1=int(input("Enter Capacity of jug 1:"))
j2=int(input("Enter Capacity of jug 2:"))
goal=int(input("Enter Target Amount:"))

visited=set()

def dfs(x,y,path):
    if x==goal or y==goal:
        path.append((x,y))
        print("\nGoal achieved Successfully!")
        print("Solution Path:")
        for state in path:
            print(state)
        return True

    if (x,y) in visited:
        return False
    visited.add((x,y))
    path.append((x,y))

    next_state=[
        (j1,y),
        (x,j2),
        (0,y),
        (x,0),

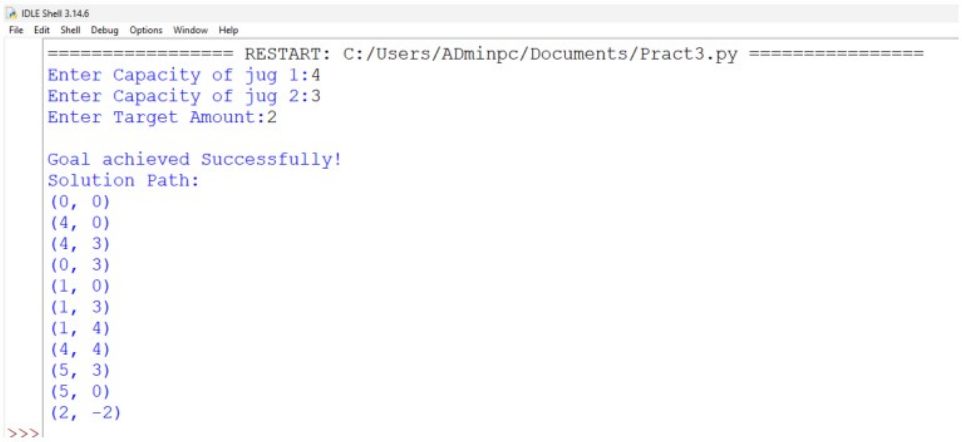
        (x-min(x,j2-y),
        y+min(x,j2-x)),

        (x+min(y,j1-y),
        y-min(y,j1-x))
    ]

    for state in next_state:
        if dfs(state[0],state[1],path.copy()):
            return True
    return False
if not dfs(0,0,[]):
        print("No solution found")

```

Output:


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
========== RESTART: C:/Users/ADminpc/Documents/Pract3.py ==========
Enter Capacity of jug 1:4
Enter Capacity of jug 2:3
Enter Target Amount:2
Goal achieved Successfully!
Solution Path:
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(1, 0)
(1, 3)
(1, 4)
(4, 4)
(5, 3)
(5, 0)
(2, -2)
>>>

Practical 4

Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS.]

Code:

```
import heapq

graph = {
   'Mumbai': [('Pune', 150), ('Hyderabad', 170)],
   'Pune': [('Mumbai', 150), ('Banglore', 840)],
   'Hyderabad': [('Mumbai', 710), ('Banglore', 570)],
   'Banglore': []
}

def ucs(start, goal):
   queue = [(0, start, [start])]
   visited = []

   while queue:
      cost, city, path = heapq.heappop(queue)

      if city == goal:
         return cost, path

      if city not in visited:
         visited.append(city)

         for neighbor, distance in graph[city]:
            heapq.heappush(
               queue,
               (cost + distance,
                neighbor,
                path + [neighbor])
            )

   return None


cost, path = ucs("Mumbai", "Banglore")

print("Least-Cost path:")
```

print("Path:", "->".join(path))
print("Total Distance:", cost, "km")

```
File  Edit  Format  Run  Options  Window  Help
import heapq
graph={
    'Mumbai':[('Pune',150),('Hyderabad',170)],
    'Pune':[('Mumbai',150),('Banglore',840)],
    'Hyderabad':[('Mumbai',710),('Banglore',570)],
    'Banglore':[]
    }
def ucs(start, goal):
    queue=[(0,start,[start])]
    visited=[]
    while queue:
        cost, city, path = heapq.heappop(queue)
        if city==goal:
            return cost,path
        if city not in visited:
            visited.append(city)
            for neighbor, distance in graph[city]:
                heapq.heappush(
                    queue,
                    (cost+distance,
                     neighbor,
                     path+[neighbor])
                    )
cost, path = ucs("Mumbai","Banglore")
print("Lease-Cost path:")
print("Path:","->".join(path))
print("Total Distance:",cost,"km")

```

Output:

```
IDLE Shell 3.10.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informat
>>>
========== RESTART: C:/Users/PC90/Desktop/AI/Practical4.py ===
Least-Cost path:
Path: Mumbai->Pune->Banglore
Total Distance: 990 km
>>>
```

Practical 5

Given an initial configuration of the 8-Puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan distance heuristic.

Code:

```
import heapq

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()


def heuristic(state, goal):
    distance = 0
    for num in range(1, 9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance


def get_neighbour(state):
    neighbour = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]

    for move in moves:
        new = blank + move

        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue

            if blank % 3 == 2 and move == 1:
                continue

            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
```

```
            neighbour.append(tuple(temp))

    return neighbour


def greedy(initial, goal):
    queue = []

    heapq.heappush(
        queue,
        (heuristic(initial, goal), initial, [])
    )

    visited = set()

    while queue:
        h, state, path = heapq.heappop(queue)

        if state == goal:
            return path + [state]

        if state not in visited:
            visited.add(state)

            for next_state in get_neighbour(state):
                heapq.heappush(
                    queue,
                    (
                        heuristic(next_state, goal),
                        next_state,
                        path + [state]
                    )
                )

    return None


initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

goal = (
```

```
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("Initial State")
print_state(initial)

print("Goal State")
print_state(goal)

solution = greedy(initial, goal)

print("Solution:")

if solution:
    for step, state in enumerate(solution):
        print("Step", step)
        print_state(state)

    print("Goal reached successfully.")
else:
    print("No solution found.")
```

File Edit Format Run Options Window Help

```python
import heapq

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()


def heuristic(state, goal):
    distance = 0
    for num in range(1, 9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance


def get_neighbour(state):
    neighbour = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]

    for move in moves:
        new = blank + move

        if new >= 0 and new < 9:
            if blank % 3 == 0 and move == -1:
                continue
            if blank % 3 == 2 and move == 1:
                continue

            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbour.append(tuple(temp))

    return neighbour


def greedy(initial, goal):
    queue = []
    heapq.heappush(
        queue,
        (heuristic(initial, goal), initial, [])
    )

    visited = set()

    while queue:
        h, state, path = heapq.heappop(queue)

        if state == goal:
            return path + [state]

        if state not in visited:
            visited.add(state)
```

```
            for next_state in get_neighbour(state):
                heapq.heappush(
                    queue,
                    (
                        heuristic(next_state, goal),
                        next_state,
                        path + [state]
                    )
                )

    return None


initial = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)

goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

print("Initial State")
print_state(initial)

print("Goal State")
print_state(goal)

solution = greedy(initial, goal)

print("Solution:")

if solution:
    for step, state in enumerate(solution):
        print("Step", step)
        print_state(state)

    print("Goal reached successfully.")
else:
    print("No solution found.")

```

Output:

```
>>>
================== RESTART: C:/Users/PC90/Desktop/AI/Pract5.py ==================
Initial State
1 2 3
4 0 6
7 5 8

Goal State
1 2 3
4 5 6
7 8 0

Solution:
Step 0
1 2 3
4 0 6
7 5 8

Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached successfully.
>>>
```

Activate Windows
Go to Settings to activate Windows.

Practical 7

Given a weighted graph representing cities and distances between them, write a Python program to find the *shortest path from Arad to Bucharest using A Search with a heuristic function (straight-line distance to destination).

Code:

```
import heapq

graph = {
   'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
   'Zerind': [('Arad', 75), ('Oradea', 71)],
   'Oradea': [('Zerind', 71), ('Sibiu', 151)],
   'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
   'Timisoara': [('Arad', 118), ('Lugoj', 111)],
   'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
   'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
   'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
   'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
   'Rimnicu Vilcea': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
   'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
   'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
   'Bucharest': []
}

heuristic = {
   'Arad': 366,
   'Zerind': 374,
   'Oradea': 380,
   'Sibiu': 253,
   'Timisoara': 329,
   'Lugoj': 244,
   'Mehadia': 241,
   'Drobeta': 242,
   'Craiova': 160,
   'Rimnicu Vilcea': 193,
   'Fagaras': 176,
   'Pitesti': 100,
   'Bucharest': 0,
}

def astar(start, goal):
```

```
    queue = []
    heapq.heappush(queue, (heuristic[start], 0, start, [start]))
    visited = set()

    while queue:
        f_cost, g_cost, city, path = heapq.heappop(queue)

        if city == goal:
            return g_cost, path

        if city not in visited:
            visited.add(city)

            for neighbor, distance in graph[city]:
                new_g = g_cost + distance
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    queue,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None, []


cost, path = astar('Arad', 'Bucharest')

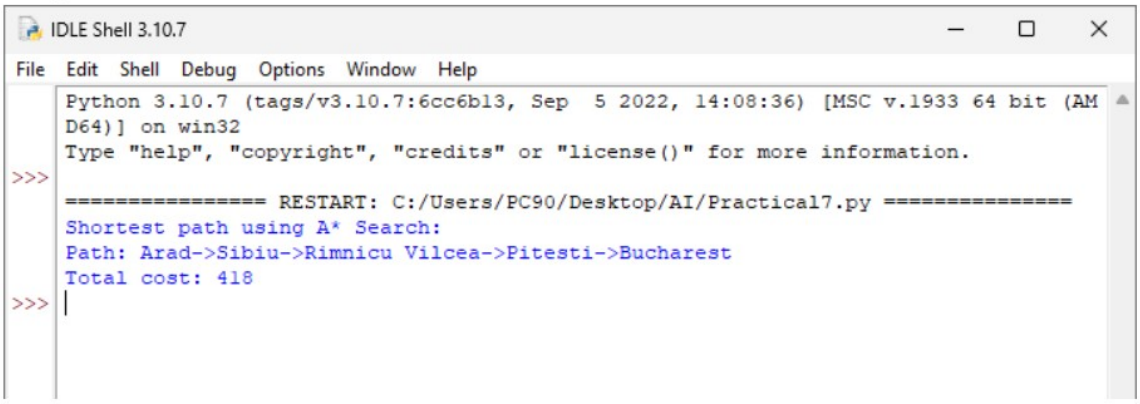
print("Shortest path using A* Search:")
print("Path:", "->".join(path))
print("Total cost:", cost)
```

```python
import heapq
graph={
    'Arad':[('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind':[('Arad', 75), ('Oradea', 71)],
    'Oradea':[('Zerind', 71), ('Sibiu', 151)],
    'Sibiu':[('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Timisoara':[('Arad', 118),('Lugoj', 111)],
    'Lugoj':[('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia':[('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta':[('Mehadia', 75), ('Craiova', 120)],
    'Craiova':[('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Rimnicu Vilcea':[('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
    'Fagaras':[('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti':[('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest':[]
}
heuristic={
    'Arad':366,
    'Zerind':374,
    'Oradea':380,
    'Sibiu':253,
    'Timisoara':329,
    'Lugoj':244,
    'Mehadia':241,
    'Drobeta':242,
    'Craiova':160,
    'Rimnicu Vilcea':193,
    'Fagaras':176,
    'Pitesti':100,
    'Bucharest':0,
}
def astar(start, goal):
    queue=[]
    heapq.heappush(queue,(heuristic[start],0,start,[start]))
    visited=set()
    while queue:
        f_cost,g_cost,city,path=heapq.heappop(queue)
        if city == goal:
            return g_cost,path
        if city not in visited:
            visited.add(city)
            for neighbor, distance in graph[city]:
                new_g = g_cost+distance
                new_f = new_g+heuristic[neighbor]


                heapq.heappush(queue,(new_f, new_g, neighbor, path+[neighbor]))


    return None,[]
cost, path=astar('Arad', 'Bucharest')
print("Shortest path using A* Search:")
print("Path:","->".join(path))
print("Total cost:",cost)
```

Output:



Practical 8

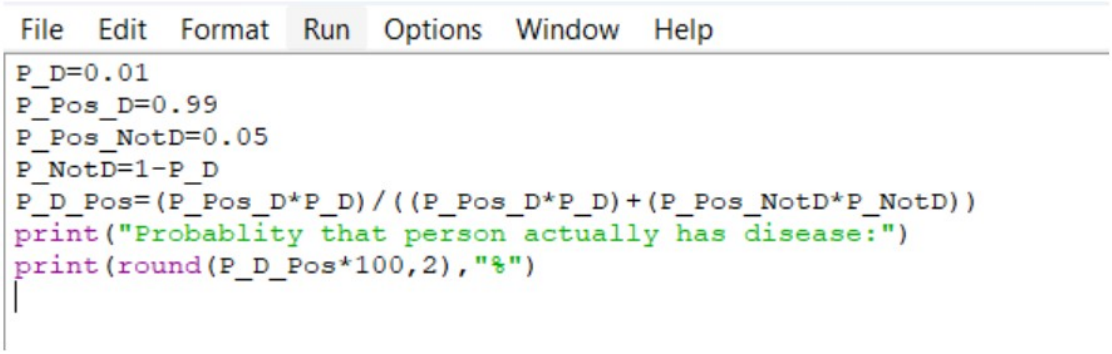
A medical test is used to detect a particular disease. Write a Python program to compute the probability that the person actually has the disease given that the test result is positive.

Code:

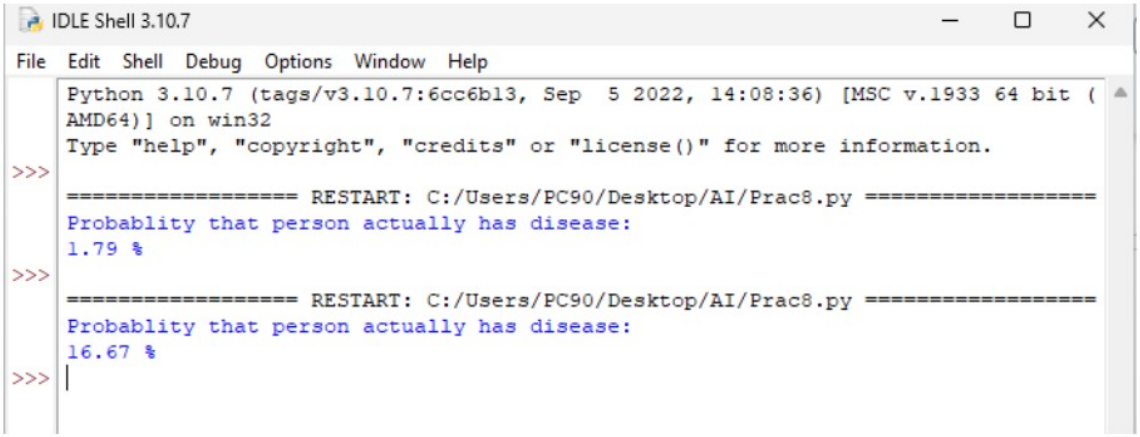
```
P_D = 0.01
P_Pos_D = 0.99
P_Pos_NotD = 0.05
P_NotD = 1 - P_D

P_D_Pos = (P_Pos_D * P_D) / ((P_Pos_D * P_D) + (P_Pos_NotD * P_NotD))

print("Probability patient actually has disease:")
print(round(P_D_Pos * 100, 2), "%")
```



Output:



Practical 9

Using a small dataset (e.g., weather or spam dataset) implement a Naive Bayes classifier using Scikit-learn.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

data = {
   'Outlook':['Sunny','Sunny','Overcast','Rainy','Rainy','Rainy','Overcast'],
   'Temperature':['Hot','Hot','Hot','Mild','Cool','Cool','Cool'],
   'Humidity':['High','High','High','High','Normal','Normal','Normal'],
   'Windy':['False','True','False','False','False','True','True'],
   'Play':['No','No','Yes','Yes','Yes','No','Yes']
}
df=pd.dataFrame(data)

le=LabelEncoder()
for col in df.columns:
   df[col]=le.fit_transform(df[col])

X=df[['Outlook','Temperature','Humidity','Windy']]
y=df['Play']

model= GaussianNB()
model.fit(X,y)

test = pd.DataFrame(
   [[2,0,0,1]],
   columns=['Outlook','Temperature','Humidity','Windy']
)

prediction=model.predict(test)
if prediction[0]==1:
   print("Prediction:Yes, we can play")
else:
   print("Prediction:No, we cannot play")
```

sgg.py - C:/Users/Admin/Desktop/sgg.py (3.12.6)

File Edit Format Run Options Window Help

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

data = {
    'Outlook':['Sunny','Sunny','Overcast','Rainy','Rainy','Rainy','Overcast'],
    'Temperature':['Hot','Hot','Hot','Mild','Cool','Cool','Cool'],
    'Humidity':['High','High','High','High','Normal','Normal','Normal'],
    'Windy':['False','True','False','False','False','True','True'],
    'Play':['No','No','Yes','Yes','Yes','No','Yes']
}
df=pd.dataFrame(data)

le=LabelEncoder()
for col in df.columns:
    df[col]=le.fit_transform(df[col])

X=df[['Outlook','Temperature','Humidity','Windy']]
y=df['Play']

model= GaussianNB()
model.fit(X,y)

test = pd.DataFrame(
    [[2,0,0,1]],
    columns=['Outlook','Temperature','Humidity','Windy']
)

prediction=model.predict(test)
if prediction[0]==1:
    print("Prediction:Yes, we can play")
else:
    print("Prediction:No, we cannot play")
```

Output:

```
>>>
================ RESTART: C:/Users/Asus/OneDrive/Desktop/Python99/pkji.py ================
Prediction:No, we cannot play
>>>
```

Practical 10

Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# P(Earthquake)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95]  # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
```

```
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90]  # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30],  # MaryCalls = False
        [0.01, 0.70]   # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,Ä
    cpd_john,
    cpd_mary
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
```

```
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
```

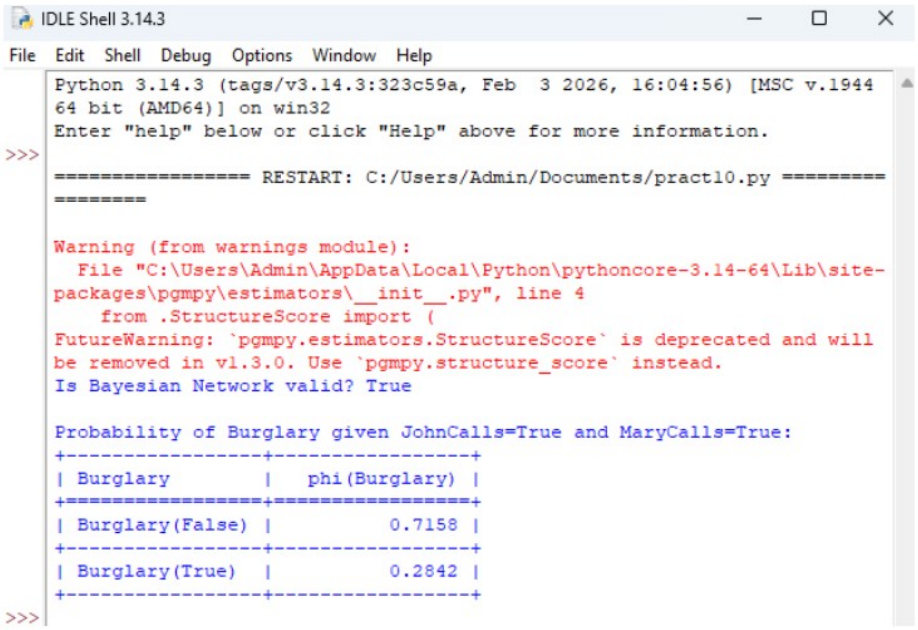
sgg.py - C:/Users/Admin/Desktop/sgg.py (3.12.6)

File Edit Format Run Options Window Help

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# P(Burglary)
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
# P(Earthquake)
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95]  # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90]  # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30],  # MaryCalls = False
        [0.01, 0.70]   # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,Ä
    cpd_john,
    cpd_mary
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
```

Name: Siddesh Shirke Roll No:2024TT44 Class: TYBSc IT

Output:


IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Admin/Documents/pract10.py ==========
Warning (from warnings module):
File "C:\Users\Admin\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pgmpy\estimators__init__.py", line 4
from .StructureScore import (
FutureWarning: `pgmpy.estimators.StructureScore` is deprecated and will be removed in v1.3.0. Use `pgmpy.structure_score` instead.
Is Bayesian Network valid? True
Probability of Burglary given JohnCalls=True and MaryCalls=True:
Burglary	phi(Burglary)
Burglary(False)	0.7158
Burglary(True)	0.2842
>>>